

MySQL分区表

一、概述

1、基本概念

在 MySQL 中，**分区表 (Partitioned Table)** 是一种将大表拆分为多个小表的技术，以提高查询性能和管理效率。这些小表称为“分区”。每个分区都可以独立于其他分区进行存储、备份、索引和其他操作。在实际应用的时候，只对这一张大表进行操作（区分分库分表，在应用上控制访问哪一张分表）。

2、分区方式/类型

MySQL支持几种不同类型的分区方式，包括RANGE、LIST、HASH和KEY：

- RANGE分区：基于列的值范围将数据分配到不同的分区。例如，可以根据日期范围将数据分配到不同的月份或年份的分区中。
- LIST分区：类似于RANGE分区，但LIST分区是基于列的离散值集合来分配数据的。可以指定一个枚举列表来定义每个分区的值。
- HASH分区：基于用户定义的表达式的哈希值来分配数据到不同的分区。这种分区方式适用于确保数据在各个分区之间均匀分布。
- KEY分区：类似于HASH分区，但KEY分区支持计算一列或多列的哈希值来分配数据。它支持多列作为分区键，并且提供了更好的数据分布和查询性能。

3、特点

1、存储方式

对于非分区表来说，如果没有开启 `innodb_file_per_table` 选项，InnoDB 存储引擎会使用 **系统表空间 (System Tablespace)** 来存储所有表的数据和索引。如果开启 `innodb_file_per_table` 选项，则每个表有独立的 `.ibd` 文件，InnoDB 存储引擎会将表的数据和索引存储在一个单独的 `.ibd` 文件中（如果开启了 `innodb_file_per_table` 选项），如表 `my_table` 的数据和索引会存储在 `my_table.ibd` 文件中。

系统表空间: 系统表空间是 InnoDB 存储引擎的默认存储方式，所有表的数据和索引都存储在共享的系统表空间文件中。系统表空间的文件通常命名为 `ibdata1`、`ibdata2` 等，位于 MySQL 的数据目录（由 `datadir` 参数指定）。当 `innodb_file_per_table` 未开启时，所有表的数据和索引都存储在系统表空间文件中。

这意味着：每个表不再有独立的 `.ibd` 文件；所有表的数据和索引都混合存储在 `ibdata1` 等文件中。

对于 **分区表**，InnoDB 存储引擎会为每个分区创建一个单独的 `.ibd` 文件，如，如果一个表有 4 个分区，那么会有 4 个对应的 `.ibd` 文件，每个文件存储一个分区的数据和索引：

假设有一个分区表 `hash_partitioned_table`，分区方式为 `HASH`，分区数量为 4，那么它的存储文件可能如下：

```
hash_partitioned_table#p#p0.ibd
hash_partitioned_table#p#p1.ibd
hash_partitioned_table#p#p2.ibd
hash_partitioned_table#p#p3.ibd
```

- `p0`、`p1`、`p2`、`p3` 是分区的名称

- 每个 `.ibd` 文件对应一个分区的数据

InnoDB 存储引擎的默认存储方式在 MySQL 的不同版本中有所演变

- MySQL 5.5 及之前版本

默认存储方式：系统表空间（System Tablespace）

特点：

- 所有表的数据和索引都存储在共享的系统表空间文件中（通常是 `ibdata1`）。
- 没有为每个表创建独立的 `.ibd` 文件。
- 删除表时，系统表空间不会自动释放空间，导致文件越来越大。

- MySQL 5.6 及之后版本

默认存储方式：每个表有独立的 `.ibd` 文件（`innodb_file_per_table=ON`）

特点：

- 从 MySQL 5.6 开始，`innodb_file_per_table` 的默认值改为 `ON`。
- 每个表的数据和索引存储在独立的 `.ibd` 文件中。
- 删除表时，对应的 `.ibd` 文件会被删除，空间会被释放。
- 系统表空间仅用于存储元数据(Metadata，元数据是描述数据库、表、索引等对象的结构信息。例如，表的定义、列的类型、索引的结构等)、Undo Logs（撤销日志，是 InnoDB 用于实现事务回滚和多版本并发控制（MVCC）的关键组件）和 Change Buffer（InnoDB 用于优化对非唯一二级索引（Secondary Index）的写操作的机制）。

- MySQL 8.0 及之后版本

默认存储方式：继续沿用 MySQL 5.6 的方式，即 `innodb_file_per_table=ON`。

新增特性：

- **表空间加密：**支持对独立表空间（`.ibd` 文件）进行加密。
- **通用表空间：**允许将多个表存储在同一个表空间文件中（通过 `CREATE TABLESPACE` 实现）。
- **数据字典改进：**元数据存储 InnoDB 表中，而不是文件系统中。

MySQL 版本	默认存储方式	特点
MySQL 5.5 及之前	系统表空间 (<code>innodb_file_per_table=OFF</code>)	所有表共享 <code>ibdata1</code> 文件，删除表不释放空间。
MySQL 5.6 及之后	独立表空间 (<code>innodb_file_per_table=ON</code>)	每个表有独立的 <code>.ibd</code> 文件，删除表时会释放空间。
MySQL 8.0 及之后	独立表空间 (<code>innodb_file_per_table=ON</code>)	支持表空间加密和通用表空间，元数据存储 InnoDB 表中。

通过以下 SQL 查询当前的 `innodb_file_per_table` 设置

```
SHOW VARIABLES LIKE 'innodb_file_per_table';
```

从物理存储与逻辑分割上说明：

- 物理上，每个分区可以存储在不同的文件或目录中，这取决于分区类型和配置。
- 逻辑上，表数据根据分区键的值被分割到不同的分区里。

2、查询性能

当执行查询时，MySQL能够确定哪些分区包含相关数据，并只在这些分区上进行搜索。这减少了需要搜索的数据量，从而提高了查询性能。对于范围查询或特定值的查询，分区可以显著减少扫描的数据量。

详情查看：补充中的mysql如何查询分区表（对查询速率上的优化）

3、数据管理与维护

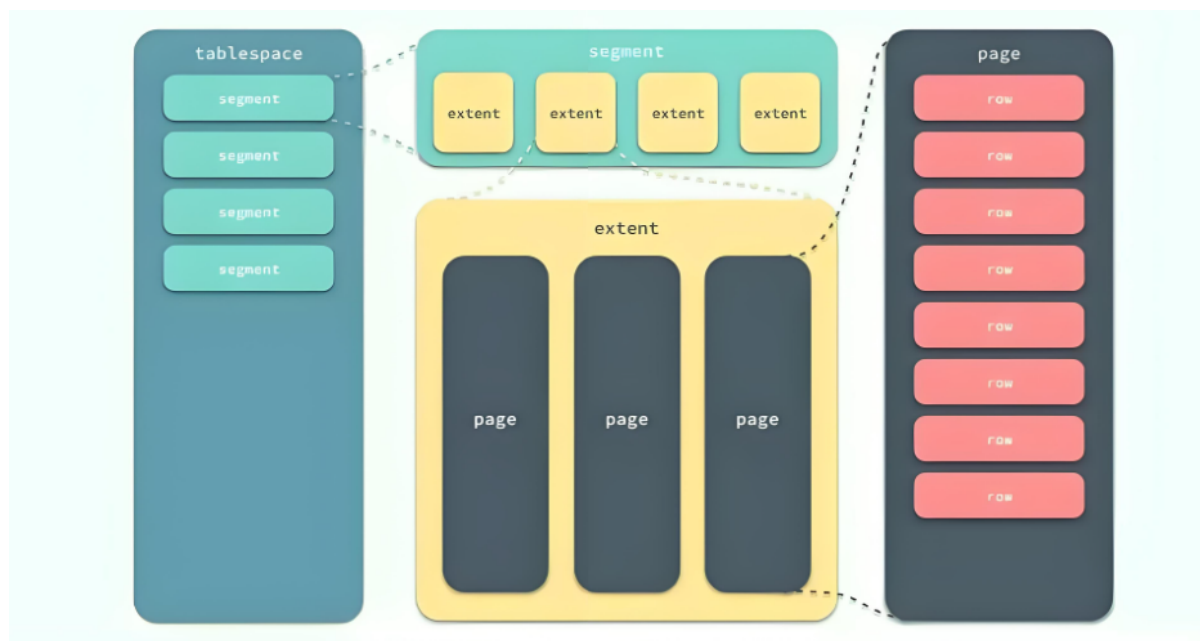
因为根据分区的存储可以知道，InnoDB 存储引擎会为每个分区创建一个单独的 `.ibd` 文件(这些分区在物理上是独立的，可以单独处理，也可以作为整体处理,这意味着数据和索引都存储在各自的分区内，MySQL尚不支持全局分区索引)，分区可以使得数据管理更加灵活。例如，可以独立地备份、恢复或优化某个分区，而无需对整个表进行操作，对于具有时效性的数据，可以通过删除或归档某个分区来快速释放存储空间。

4、扩展与并行处理

分区技术使得数据库表更容易扩展到更大的数据集。当表的大小超过单个存储设备的容量时，可以使用分区将数据分布到多个存储设备上。由于每个分区可以独立处理，因此可以并行执行查询和其他数据库操作，从而进一步提高性能。

4、InnoDB逻辑存储结构

InnoDB存储引擎的逻辑结构是一个层次化的体系，主要由表空间、段、区和页构成



- 表空间：是InnoDB数据的最高层容器，所有数据都逻辑地存储在这里。
- 段（Segment）：是表空间的重要组成部分，根据用途可分为数据段、索引段和回滚段等。InnoDB引擎负责管理这些段，确保数据的完整性和高效访问。
- 区（Extent）：由连续的页组成，每个区默认大小为1MB，不论页的大小如何变化。为保证页的连续性，InnoDB会一次性从磁盘申请多个区。每个区包含64个连续的页，当默认页大小为16KB时。在段开始时，InnoDB会先使用32个碎片页存储数据，以优化小表或特定段的空间利用率（详情查看补充：关于段分配存储空间前中先使用32个碎片页存储数据后，再使用区进行分配）。

- 页 (Page)：是InnoDB磁盘管理的最小单元，也被称为块。其默认大小为16KB，但可通过配置参数进行调整。页的类型多样，包括数据页、undo页、系统页等，每种页都有其特定的功能和结构。

5、注意事项

- 分区键选择：选择合适的分区键至关重要。确保分区键能够均匀地分布数据，并且与查询条件相匹配，以提高查询性能。
- 分区数量限制：MySQL对单个表的分区数量有限制（通常为1024个分区）。在设计分区策略时要考虑这个限制
- 性能和资源消耗：虽然分区可以提高性能，但在某些情况下，过多的分区可能导致额外的性能和资源消耗。因此，要合理设计分区策略以平衡性能和资源消耗（最直白的就是分区多了，我加一条数据的时候就需要去考虑进入哪一个分区下，即随着分区数量的增多，维护的成本增大）
- 兼容性和迁移：在迁移现有表到分区表之前，要确保备份原始数据并测试迁移过程的正确性。此外，要了解不同MySQL版本之间对分区功能的支持和兼容性差异
- 全局唯一索引限制：全局唯一索引不受分区的限制，它要求所有分区合并后的数据是唯一的。在分区表上创建全局唯一索引时存在限制。确保了解这些限制，并根据需要进行调整。
 - **跨分区的唯一性要求**：全局唯一索引要求表中的每一行数据在整个表的范围内具有唯一性，而不仅仅是在单个分区内
 - **性能问题**：一索引会带来额外的性能开销，尤其是在大规模分区表中。因为MySQL需要检查每个分区的数据，并确保在所有分区内的索引值是唯一的。当插入、更新或删除数据时，MySQL必须确保跨所有分区的索引值一致性，这可能会显著降低写入操作的速度
 - **分区表上的唯一性限制**：在一些分区类型（如 RANGE 分区）中，可能不允许创建全局唯一索引。这是因为，某些分区类型本身并不支持跨所有分区的全局唯一性。在 RANGE 分区中，数据根据某个范围值（例如日期、数字等）划分到不同的分区。每个分区内部的数据是独立存储的，可能不同分区，都有id=1的数据，因此 MySQL 在每个分区内部对索引列进行唯一性约束。但是，跨多个分区进行唯一性检查会变得非常复杂，因为数据可以分布在多个分区中，MySQL 需要跨分区进行检查，这会导致性能问题，也不符合一些分区类型的设计原则，如对 id 创建唯一索引。
 - **分区策略的设计**：如果你在分区表上使用全局唯一索引，你需要仔细选择分区策略。例如，如果你选择按日期分区，那么每个分区可能会包含不同时间段的数据，但全局唯一索引要求这些数据在所有时间段之间也要唯一

6、其他参考

- 详解与实践：https://blog.csdn.net/qg_26664043/article/details/138452285
- 底层实现：<https://blog.csdn.net/xushiyu1996818/article/details/103859998>
- <https://blog.csdn.net/angryshan/article/details/120024460>

二、补充

1、关于段分配存储空间前中先使用32个碎片页存储数据后，再使用区进行分配

1.InnoDB 的存储结构可以分为以下几个层次：

1. 表空间 (Tablespace)：

- 表空间是 InnoDB 存储引擎的最高层逻辑存储结构。
- 每个表空间由多个段 (Segment) 组成。

2. 段 (Segment) :

- 段是表空间的子结构，通常分为数据段、索引段和回滚段。
- 每个段由多个区 (Extent) 组成。

3. 区 (Extent) :

- 区是段的子结构，由连续的页 (Page) 组成。
- 每个区默认大小为 1MB，包含 64 个连续的页 (默认页大小为 16KB)。

4. 页 (Page) :

- 页是 InnoDB 的最小存储单元，默认大小为 16KB。
- 页用于存储数据、索引等信息。

2.关于碎片页 (Fragmented Pages) 的作用:

在段 (Segment) 的初始阶段，InnoDB 会先使用 **32 个碎片页** (碎片页不一定是连续的，这些单独的页可能分散在表空间不同位置)来存储数据。

- **碎片页**：这些页不是从完整的区 (Extent) 中分配的，而是单独分配的页。
- **32 个碎片页**：在段开始时，InnoDB 会先分配 32 个单独的页 (而不是直接分配一个完整的区)。
- **优化小表或特定段的空间利用率**：对于小表或某些特定的段 (如索引段)，直接分配一个完整的区 (1MB) 可能会导致空间浪费。通过先使用碎片页，可以更灵活地管理空间，避免浪费。

3.为什么使用碎片页

使用碎片页的主要目的是 **优化空间利用率**，具体原因如下：

1. 小表的存储需求：

- 对于小表，数据量可能远小于 1MB。如果直接分配一个完整的区，会导致大量空间浪费。
- 使用碎片页可以按需分配存储空间，避免浪费。

2. 特定段的需求：

- 某些段 (如索引段) 的初始数据量可能较小，直接分配完整的区不划算。
- 使用碎片页可以更灵活地管理这些段的空间。

3. 性能优化：

- 在段开始时，数据量通常较小。使用碎片页可以减少初始分配的开销。
- 当数据量增长到一定程度时，InnoDB 会自动切换到分配完整的区。

4.碎片页的工作机制

1. 初始阶段：

- 在段开始时，InnoDB 会先分配 32 个单独的页 (碎片页)。
- 这些页不一定是连续的，但它们可以满足小表或特定段的初始存储需求。

2. 数据增长阶段：

- 当碎片页用完后，InnoDB 会开始分配完整的区 (每个区包含 64 个连续的页)。
- 这样可以保证后续的数据存储是连续的，提高性能。

3. 空间回收：

- 如果数据被删除，InnoDB 会回收空闲的页。
- 回收的页可以重新用于存储新数据，避免空间浪费。

关于InnoDB 存储引擎中对空闲页的回收

已占用的页（即仍然包含有效数据的页）不会被回收，除非这些页中的数据被删除或移动到其他位置。如果某个页中的所有行都被删除或移动到其他位置，InnoDB 会将对应的页标记为“空闲”（Free），但这些页并不会立即被释放回操作系统或表空间，这些空闲页会被保留在 InnoDB 的**空闲列表（Free List）**中，供后续的插入操作重用。如果后续的插入操作需要空间，InnoDB 会优先使用这些空闲页，而不是分配新的页。如果表中存在大量的删除操作，可能会导致许多页被标记为空闲，但这些页仍然属于该表，不会立即释放回表空间。如果需要释放空间回操作系统，可以执行 `OPTIMIZE TABLE` 命令。这个操作会重建表，并将未使用的空间释放回操作系统（注意：`OPTIMIZE TABLE` 是一个代价较高的操作，因为它会重建表并锁定表），通过 `OPTIMIZE TABLE` 或 `ALTER TABLE ... ENGINE=InnoDB` 可以整理碎片，使数据页更加紧凑。

5. 示例说明

假设有一个小表，初始数据量只有 100KB：

- 如果直接分配一个完整的区（1MB），会导致 900KB 的空间浪费。
- 使用碎片页后，InnoDB 只会分配 7 个页（ $100\text{KB} / 16\text{KB} \approx 7$ 页），避免了空间浪费。

当表的数据量增长到 2MB 时：

- InnoDB 会先使用碎片页存储部分数据。
- 当碎片页用完后，InnoDB 会开始分配完整的区（每个区 1MB），保证数据的连续性。

6. 总结

- **碎片页**：在段开始时，InnoDB 会先分配 32 个单独的页，而不是直接分配完整的区。
- **优化空间利用率**：碎片页可以避免小表或特定段的空间浪费。
- **数据增长后的处理**：当碎片页用完后，InnoDB 会切换到分配完整的区，保证数据的连续性。

2、关于Hash和Key类型的分区的使用

在 MySQL 中，`HASH` 分区和 `KEY` 分区是两种常见的分区方式，它们基于哈希算法将数据分布到不同的分区中。以下是它们的使用示例和详细说明：

HASH分区：

`HASH` 分区基于用户指定的表达式（通常是列值）计算哈希值，然后将数据分布到不同的分区中。适用于均匀分布数据的场景。

```
-- 创建一个使用 HASH 分区的表
CREATE TABLE hash_partitioned_table (
    id INT NOT NULL,
    name VARCHAR(100),
    created_at DATE
)
PARTITION BY HASH (id) -- 使用 id 列进行哈希分区
PARTITIONS 4; -- 分成 4 个分区

-- 插入数据
INSERT INTO hash_partitioned_table (id, name, created_at)
VALUES (1, 'Alice', '2023-01-01'),
       (2, 'Bob', '2023-02-01'),
```

```
(3, 'Charlie', '2023-03-01'),
(4, 'David', '2023-04-01');

-- 查询数据
SELECT * FROM hash_partitioned_table WHERE id = 2;
```

说明：

- `HASH` 分区使用 `PARTITION BY HASH (expr)` 语法，其中 `expr` 是一个表达式（通常是列名）。

表达式类型可以是如下形式：

- **单个列名**：例如 `id`、`created_at`。
- **列的组合**：例如 `(id, created_at)`。
- **函数或计算表达式**：例如 `YEAR(created_at)`、`id % 10`

注意事项：

- 表达式的结果必须是**整数值**，因为哈希函数需要整数输入。
 - 表达式应尽量保证数据的均匀分布，避免某些分区数据过多或过少。
 - 如果表达式过于复杂，可能会影响性能。
- 分区数量由 `PARTITIONS` 指定。
 - 数据分布基于哈希值，因此分区键的选择应尽量均匀

在 MySQL 的 `HASH` 分区中，数据具体分配到哪个分区是由 MySQL 内部的哈希函数决定的。哈希函数会对分区键（在你的例子中是 `id` 列）的值进行计算，然后根据计算结果将数据分配到对应的分区。

分区分配规则：

MySQL 使用以下公式计算分区编号

```
分区编号 = MOD(哈希函数(id), 分区数量)
```

其中：

- `哈希函数(id)` 是 MySQL 内部的一个哈希函数，具体实现细节对用户透明。
- `分区数量` 是你在 `PARTITIONS` 子句中指定的值（在你的例子中是 4）。

分区编号从 0 开始，因此分区编号范围是 0 到 `分区数量 - 1`。

在你的例子中：

- 分区键是 `id`。
- 分区数量是 4。
- 插入的数据是 `id = 2`。

假设 MySQL 的哈希函数对 `id = 2` 的计算结果是 `HASH(2) = x`，那么分区编号为：

```
分区编号 = MOD(x, 4)
```

由于哈希函数的具体实现是 MySQL 内部的，我们无法直接知道 `x` 的值。但可以通过实验或查询分区信息来确定。

KEY分区：

KEY 分区类似于 HASH 分区，但它使用 MySQL 内部的哈希函数，并且分区键可以是整数或字符串。如果未指定分区键，MySQL 会默认使用主键作为分区键。

示例：

```
-- 创建一个使用 KEY 分区的表
CREATE TABLE key_partitioned_table (
    id INT NOT NULL,
    name VARCHAR(100),
    created_at DATE
)
PARTITION BY KEY (id) -- 使用 id 列进行 KEY 分区
PARTITIONS 4; -- 分成 4 个分区

-- 插入数据
INSERT INTO key_partitioned_table (id, name, created_at)
VALUES (1, 'Alice', '2023-01-01'),
       (2, 'Bob', '2023-02-01'),
       (3, 'Charlie', '2023-03-01'),
       (4, 'David', '2023-04-01');

-- 查询数据
SELECT * FROM key_partitioned_table WHERE id = 3;
```

说明：

- KEY 分区使用 PARTITION BY KEY (column_list) 语法。
- 如果未指定分区键，MySQL 会使用主键。
- 分区数量由 PARTITIONS 指定。

HASH分区与KEY分区区别：

特性	HASH 分区	KEY 分区
分区键	用户指定的表达式（如列名）	列名（默认使用主键）
哈希函数	用户定义的表达式	MySQL 内部的哈希函数
适用列类型	整数、日期等	整数、字符串等
是否支持多列	是	是
分区数量	由 PARTITIONS 指定	由 PARTITIONS 指定

- HASH 分区和 KEY 分区都是基于哈希算法的分区方式，适用于需要均匀分布数据的场景。
- HASH 分区需要指定一个表达式，而 KEY 分区使用 MySQL 内部的哈希函数。
- 分区表的设计需要根据实际业务需求和数据分布特点进行优化。

3、分区列 (Partitioning Key) 的选择

MySQL 为什么要求分区列必须是主键或唯一键的一部分，如果分区列不是主键或唯一键的一部分，MySQL 会报错。为什么呢？

- **数据唯一性**：分区表的每个分区相当于一个独立的子表（意味着不同子表没有唯一约束的校验）。如果分区列不是主键或唯一键的一部分，可能会导致数据在多个分区中重复，破坏数据的唯一性。

具体示例

假设有一个表 `orders`，包含以下列：

- `order_id` (订单ID, 主键)
- `customer_id` (客户ID)
- `order_date` (订单日期)

将该表按 `customer_id` 分区，即数据会根据不同的客户ID分配到不同的分区上

```
CREATE TABLE orders (  
  order_id INT,  
  customer_id INT,  
  order_date DATE,  
  PRIMARY KEY (order_id)  
)  
PARTITION BY RANGE (customer_id);
```

假设我们按 `customer_id` 进行分区，但 `customer_id` 并不是主键的一部分。以下是可能发生的情况：

1. 订单表分为多个分区，例如 `customer_id` 为 1 的数据在分区1，`customer_id` 为 2 的数据在分区2等。
2. 如果我们在不同的分区上插入相同的 `order_id`，但是它们对应不同的 `customer_id`，就会出现重复数据，破坏了订单数据的唯一性。

例如，在分区1 (`customer_id = 1`) 插入以下数据：

```
sqlCopy CodeINSERT INTO orders (order_id, customer_id, order_date)  
VALUES (1001, 1, '2025-03-03');
```

然后在分区2 (`customer_id = 2`) 插入以下数据：

```
sqlCopy CodeINSERT INTO orders (order_id, customer_id, order_date)  
VALUES (1001, 2, '2025-03-03');
```

此时，我们发现 `order_id = 1001` 在不同的分区中重复出现，尽管 `order_id` 在整个表中应该是唯一的，因为我们没有在分区列 (`customer_id`) 上设置唯一约束或将其包含在主键中，导致了数据重复。

比方说我一个表的唯一键是a,b，我分区按照a列进行分，那么相同的a肯定会在同一个分区下，同一个分区中，就可以保证该分区中数据的唯一性，从而保证整张表的数据的唯一。

每个分区 **不独立约束** `a` 和 `b` 的唯一性，唯一性检查是跨所有分区进行的，虽然你根据 `a` 进行分区，但复合主键的唯一性约束会跨越所有分区进行检查，确保整个表中 `a` 和 `b` 的组合是唯一的。每个分区只是根据 `a` 字段决定数据的存储位置，并没有独立对 `a`, `b` 进行唯一性检查。唯一性约束是表级的，而不是分区级的

△ 可以试一下，这么创建分区是否会报错

- **查询优化**：分区列是主键或唯一键的一部分（又能进行分区剪枝又可以用上各个分区的索引），可以帮助 MySQL 优化查询，快速定位到具体的分区。在进行基于主键或唯一键的查询时，如果分区列不是主键或唯一键一部分，MySQL可能需要在所有分区中进行搜索，从而降低了查询性能
- **数据一致性**：确保数据在分区之间是唯一的，避免数据冲突。当表被分区时，每个分区实际上可以看作是一个独立的“子表”。如果分区列不是主键或唯一键的一部分，那么在执行更新或删除操作时，MySQL需要确保跨所有分区的数据一致性，这会增加操作的复杂性和开销。
- **分区策略**：MySQL的分区策略是基于分区列的值来将数据分配到不同的分区中。如果分区列不是主键或唯一键的一部分，那么分区策略可能会变得复杂且低效，因为系统需要额外处理主键或唯一键的约束（即插入数据无法保存全局唯一）

具体示例：

示例 1：分区列是主键的一部分

假设有一个订单表 `orders`，我们希望根据 `order_date` 字段对表进行分区，同时 `order_id` 是主键。

```
CREATE TABLE orders (  
    order_id INT AUTO_INCREMENT,  
    order_date DATE,  
    customer_id INT,  
    amount DECIMAL(10, 2),  
    PRIMARY KEY (order_id, order_date) -- 主键包含 order_id 和 order_date  
)  
PARTITION BY RANGE (YEAR(order_date)) (  
    PARTITION p0 VALUES LESS THAN (2020),  
    PARTITION p1 VALUES LESS THAN (2021),  
    PARTITION p2 VALUES LESS THAN (2022),  
    PARTITION p3 VALUES LESS THAN (2023),  
    PARTITION p4 VALUES LESS THAN MAXVALUE  
);
```

- **分区列**： `order_date`。
- **主键**： `(order_id, order_date)`。
- **说明**： `order_date` 是主键的一部分，满足分区列必须是主键或唯一键的一部分的要求。

示例 2：分区列是唯一键的一部分

假设有一个用户日志表 `user_logs`，我们希望根据 `log_date` 字段对表进行分区，同时 `log_id` 是唯一键。

```
CREATE TABLE user_logs (  
    log_id INT AUTO_INCREMENT,  
    user_id INT,  
    log_date DATE,
```

```

    action VARCHAR(50),
    UNIQUE KEY (log_id, log_date) -- 唯一键包含 log_id 和 log_date
)
PARTITION BY RANGE (YEAR(log_date)) (
    PARTITION p0 VALUES LESS THAN (2020),
    PARTITION p1 VALUES LESS THAN (2021),
    PARTITION p2 VALUES LESS THAN (2022),
    PARTITION p3 VALUES LESS THAN (2023),
    PARTITION p4 VALUES LESS THAN MAXVALUE
);

```

- **分区列：** `log_date`。
- **唯一键：** `(log_id, log_date)`。
- **说明：** `log_date` 是唯一键的一部分，满足分区列必须是主键或唯一键的一部分的要求。

错误示例 1：分区列不是主键或唯一键的一部分

假设有一个表 `sales`，我们希望根据 `sale_date` 字段对表进行分区，但 `sale_date` 不是主键或唯一键的一部分。

```

CREATE TABLE sales (
    sale_id INT AUTO_INCREMENT PRIMARY KEY, -- 主键只有 sale_id
    sale_date DATE,
    product_id INT,
    quantity INT,
    amount DECIMAL(10, 2)
)
PARTITION BY RANGE (YEAR(sale_date)) (
    PARTITION p0 VALUES LESS THAN (2020),
    PARTITION p1 VALUES LESS THAN (2021),
    PARTITION p2 VALUES LESS THAN (2022),
    PARTITION p3 VALUES LESS THAN (2023),
    PARTITION p4 VALUES LESS THAN MAXVALUE
);

```

- **错误原因：** `sale_date` 不是主键或唯一键的一部分。
- **报错信息：** ERROR 1503 (HY000): A PRIMARY KEY must include all columns in the table's partitioning function

错误示例2： 主键是 a, b, 我分区表的分区列设置成a, c

4、mysql如何查询分区表（对查询速率上的优化）

在 MySQL 中，分区表（Partitioned Tables）能够将数据分散到多个物理分区中，提高查询效率。对于有分区的表，MySQL 会基于查询条件来决定是否可以利用分区来优化查询，尤其是当查询条件中涉及到分区列时。

比如说创建的分区如下所示：

按照 `customer_id` 列分区的订单表

```
CREATE TABLE orders (  
    order_id INT,  
    customer_id INT,  
    order_date DATE,  
    PRIMARY KEY (order_id, customer_id)  
)  
PARTITION BY RANGE (customer_id) (  
    PARTITION p0 VALUES LESS THAN (100),  
    PARTITION p1 VALUES LESS THAN (200),  
    PARTITION p2 VALUES LESS THAN (300)  
);
```

1.分区裁剪 (Partition Pruning)

执行以下查询：

```
SELECT * FROM orders WHERE customer_id = 150;
```

MySQL 会识别出这个查询涉及到 `customer_id`，并且根据分区定义的范围，直接选择分区 `p1` 来执行查询，而不需要扫描 `p0` 或 `p2` 分区。因此，这就是 **分区裁剪** 的优化效果

2.分区列的选择性

当查询条件中包含分区列时，MySQL 会尝试通过分析查询条件和分区策略来确定最优的分区执行方式。如果查询没有使用分区列进行过滤，或者分区列的选择性较低（即查询条件无法有效区分多个分区），则分区的优势可能无法充分发挥。

```
SELECT * FROM orders WHERE order_date = '2025-03-03';
```

这个查询没有涉及分区列 `customer_id`，因此 MySQL 会扫描所有的分区。这时，分区并不会带来显著的查询性能提升，因为没有应用分区裁剪。

3.范围查询 (Range Queries)

分区表的另一个优势是对 **范围查询** 的优化。如果查询条件涉及到分区列，并且查询条件符合某个范围，MySQL 会尽量减少扫描的分区范围。

```
SELECT * FROM orders WHERE customer_id BETWEEN 100 AND 200;
```

MySQL 在这个查询中可以有效地裁剪分区，只扫描 `p1` 和 `p2` 这两个分区，因为查询条件限制了 `customer_id` 的范围。

4.合并多个分区的查询 (Partition-wise Join)

如果执行连接操作时，查询涉及多个分区，MySQL 可以进行 **分部级联连接**，即在每个分区中单独进行连接操作，而不是在整个表上进行连接。这个优化方式可以显著减少查询所需的时间，特别是在大规模分区表的情况下。

假设我们有两个分区表 `orders` 和 `customers`，分别按 `customer_id` 列进行分区，如果我们进行如下连接查询：

```
SELECT *
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.customer_id = 150;
```

MySQL 会根据 `customer_id = 150` 的条件，只扫描相关分区（例如 `p1`），避免扫描所有分区，从而提高连接查询的效率。

5.总结

- **没有使用分区列的查询：**如果查询没有使用分区列（例如查询时没有涉及 `customer_id`），MySQL 将扫描所有分区，无法利用分区优化。
- **分区键的选择：**为提高查询效率，分区列的选择非常重要。通常选择高选择性且常用于查询的列作为分区列，例如经常作为过滤条件的列。
- MySQL 对分区表的查询优化主要依赖于 **分区裁剪**。当查询条件涉及到分区列时，MySQL 能够有效地只扫描与查询条件匹配的分区。为了更好地利用分区表的优势，建议选择查询时常使用的列作为分区列，特别是高选择性的列。此外，分区表对范围查询特别有利，但对没有涉及分区列的查询则效果有限。

5、关于Mysql分区和分表的区别

特性	分区 (Partitioning)	分表 (Sharding)
物理存储	数据分散到不同的分区，但属于同一张表。	数据分散到不同的表中，每张表独立存储。
逻辑视图	逻辑上是一张表，对应用程序透明。	逻辑上是多张表，应用程序需要明确选择表。
管理复杂度	管理简单，MySQL 自动处理分区逻辑。	管理复杂，需要在应用程序中实现分表逻辑。
查询性能	查询时 MySQL 自动优化分区访问。	查询时需要手动选择表，性能优化依赖实现。
适用场景	单表数据量大，但希望保持逻辑上的统一。	数据量极大，需要分布式存储和查询。

分区优点：

- **透明性：**应用程序无需关心数据分区，MySQL 自动处理分区逻辑。
- **管理方便：**可以通过分区快速删除旧数据（例如按时间分区）。
- **查询优化：**MySQL 可以只访问相关分区，减少扫描的数据量。

分表优点：

- **扩展性强：**可以将数据分布到不同的服务器上，支持分布式存储和查询。

- **灵活性高**：可以根据业务需求自定义分表逻辑。
- **性能提升**：通过分散数据，减少单表的压力。

如何选择分区还是分表：

- **选择分区**：
 - 数据量较大，但不需要分布式存储。
 - 希望保持逻辑上的统一，减少应用程序的复杂性。
 - 需要快速删除旧数据（例如按时间分区）。
- **选择分表**：
 - 数据量极大，单表或单服务器无法承受。
 - 需要分布式存储和查询。
 - 可以接受在应用程序中实现分表逻辑。

总结：

- **分区**：物理上分散存储，逻辑上是一张表，适合单表数据量大的场景。
- **分表**：物理和逻辑上都分散存储，适合数据量极大且需要分布式存储的场景。

6、关于MySQL的物理存储结构和内部逻辑结构

参考文章：

<https://blog.csdn.net/u013905744/article/details/111624539>

ibd文件：<https://blog.csdn.net/u012534547/article/details/145495300>

分区分的是innodb逻辑结构上的区吗？答案不是的。

在 MySQL 中，分区表是将大表拆分成多个逻辑上独立的子表，每个子表称为一个分区。每个分区可以有独立的数据存储（即，每个分区会有自己的 `.ibd` 文件），并且每个分区的数据都按照某种分区键进行存储。

逻辑结构关系：

- **分区表** → 包含多个分区，每个分区有自己的 `.ibd` 文件。
- **.ibd 文件** → 存储特定分区的 **段 (Segment)** 。
- **段 (Segment)** → 内部有多个 **区 (Extent)** 。
- **区 (Extent)** → 内部由多个 **页 (Page)** 组成

△ 可以去看下书里

关于MySQL中Innodb存储引擎下的物理存储结构和逻辑存储结构间的映射关系

- 表 (Table)

逻辑存储：表是用户定义的数据库对象，通常由表的名字和其结构（如字段和约束等）来标识。

物理存储：InnoDB 存储表的数据通常存储在 `.ibd` 文件中（对于使用独立表空间的表）。对于使用共享表空间（默认设置）的表，表的数据和索引存储在共享的表空间文件（如 `ibdata1`）中，而不是每个表单独一个 `.ibd` 文件。每个表无论是在独立表空间还是共享表空间下，都会包含数据和索引的物理存储。

- 段 (Segment)

逻辑存储：段是存储表数据和索引的逻辑单元，它分为多个类型，如数据段、索引段、undo段等。

物理存储：在 InnoDB 中，段是通过物理的区 (Extent) 来存储的，即段只是用来帮助理解不同类型的数据存储结构（如数据段、索引段等），而在物理实现上，**区 (Extent)** 才是实际存储数据的单位。例如，数据段用于存储表的数据行，而索引段用于存储表的索引数据。段本身并没有单独的物理表示，而是由一系列的区组成，区被分配在物理文件中。

- 区 (Extent)

逻辑存储：区是段的基本单位，它包含多个页面 (Page)，通常是 64KB（对于默认配置）。

物理存储：区在物理上由 64KB 大小的连续存储单元组成（不等同于磁盘分区）。在 `.ibd` 文件中，区的分配是连续的，可以存储多个页。区的分配是动态的，系统会根据需要分配和回收区。区中的数据是按页来存储的。

- 页 (Page)

逻辑存储：页是 InnoDB 中最小的存储单位，每个页的大小通常为 16KB（这是 InnoDB 的默认配置，且一般不可修改）。每个表的数据和索引数据都以页为单位来存储和管理。

物理存储：每个页的大小为 16KB（除非修改配置）。每个页包含一个表的数据行、索引条目等。页是存储表数据的基本单位。InnoDB 中的数据页、索引页、undo 日志页等都在物理文件中以页为单位进行存储。

尽管从逻辑上 InnoDB 的存储结构可以分为表、段、区和页，物理存储的实现也遵循这个层次结构，但在物理存储中有一些优化和实现上的细节，特别是在如何分配空间和管理内存方面。此外，具体表是否在独立表空间或共享表空间中存储也会影响实际的物理布局。

7、关于MySQL分区对分区列为NULL情况的处理

MySQL中，当涉及到分区时，系统并不会特别禁止NULL值。不论是列的实际值还是用户自定义的表达式结果，MySQL通常会将NULL值视为0进行处理。然而，这种行为可能并不总是符合数据完整性和准确性的要求。为了避免这种隐式的NULL到0的转换，最佳实践是在设计数据库表时，对相关列明确声明为“NOT NULL”。这样做可以确保数据的准确性和一致性，同时避免由于NULL值被错误地解释为0而导致的潜在问题。因此，在设计分区表时，应该谨慎考虑NULL值的处理方式，并根据需要采取相应的预防措施。

此外，如果确实需要存储NULL值，并且不希望MySQL将其视为0，可以考虑使用其他特殊值（如某个不可能在实际业务中出现的标识值）来代替NULL，或者在设计分区策略时明确考虑NULL值的处理逻辑。这样可以在保持数据完整性的同时，更好地满足业务需求

8、关于分区表是如何维护索引的以及其索引机制

mysql分区后每个分区成了独立的文件，虽然从逻辑上还是一张表其实已经分成了多张独立的表，每个独立的表都有自己的.ibd文件，用户存储数据和索引。从“information_schema.INNODB_SYS_TABLES”系统表可以看到每个分区都存在独立的TABLE_ID,由于Innodb数据和索引都是保存在".ibd"文件当中（从INNODB_SYS_INDEXES系统表中也可以得到每个索引都是对应各自的分区(primary key和unique也不例外)），所以分区表的索引也是随着各个分区单独存储。

实际上，在MySQL中，分区表**并不存在一个全局的、跨所有分区的单一索引结构**。每个分区的索引是独立维护的，而不是通过一个全局索引来管理。

分区表的索引维护方法如下：

- **插入数据**：数据会根据分区键分配到对应的分区，索引更新只发生在该分区。
- **删除数据**：只影响相关分区的索引。
- **更新数据**：如果更新涉及分区键，数据可能会移动到另一个分区，索引也会相应更新

示例1：

有一个按年份分区的表

```
CREATE TABLE sales (  
  sale_id INT,  
  sale_date DATE,  
  amount DECIMAL(10, 2),  
  INDEX idx_sale_date (sale_date)  
) PARTITION BY RANGE (YEAR(sale_date)) (  
  PARTITION p0 VALUES LESS THAN (2020),  
  PARTITION p1 VALUES LESS THAN (2021),  
  PARTITION p2 VALUES LESS THAN (2022)  
);
```

对于每个分区（如 p0、p1、p2）都有自己的 .ibd 文件，存储该分区的数据和索引（如 idx_sale_date）

执行以下查询

```
SELECT * FROM sales WHERE sale_date = '2021-05-01';
```

- MySQL会通过分区剪裁确定只需要访问分区 p1（因为 YEAR('2021-05-01') = 2021）
- 然后，MySQL会使用分区 p1 的 idx_sale_date 索引来查找数据。

示例2：

```
CREATE TABLE A (  
  a INT,  
  b INT,  
  c INT,  
  PRIMARY KEY (a, c) -- 主键包含a和c  
) PARTITION BY HASH(b) PARTITIONS 4;
```

表 A 按字段 b 进行哈希分区，分为4个分区。主键是 (a, c)，这意味着主键索引是基于 a 和 c 的组合。

执行如下查询：

```
SELECT * FROM A WHERE c = 123;
```

分区剪裁：

- 由于查询条件中没有分区键 b，MySQL无法确定具体需要访问哪个分区。
- 因此，MySQL会**扫描所有分区**（即4个分区）。

索引使用：

- 在每个分区中，MySQL会尝试使用主键索引 (a, c) 来查找满足 c = 123 的数据。
- 但由于查询条件 c = 123 不符合主键索引的最左前缀规则（缺少 a），MySQL**无法高效使用主键索引**。
- 因此，MySQL可能会退化为**全分区扫描**（即扫描每个分区的所有数据）。

此时MySQL需要扫描所有4个分区的数据，导致查询性能较差，并且由于查询条件不符合主键索引的最左前缀规则，索引无法被高效利用。

分区表局限：

由于分区表的索引是分区级别的，某些查询可能会导致性能问题：

- 如果查询条件无法有效剪裁分区（如未使用分区键），MySQL可能需要扫描所有分区的索引，导致性能下降。
- 分区表的唯一索引必须包含分区键，因为唯一性只能在分区内保证，而不能跨分区保证。（只有分区字段包含唯一索引，那么同一个分区字段的一定落在同一个分区中，同一个分区中能保证唯一，保证局部唯一，从而保证全局一定唯一）

9、分区与性能考量

技术的运用需要恰到好处才能发挥其优势。以显式锁为例，虽然功能强大，但使用不当可能导致性能下降或其他不良后果。同样地，分区技术也并非万能的性能提升工具。

分区确实可以为某些SQL查询带来性能上的提升，但其主要价值在于提高数据库的高可用性管理。在应用分区技术时，我们需要根据数据库的使用场景来谨慎选择。

数据库应用大体上可分为OLTP（在线事务处理）和OLAP（在线分析处理）两类。对于OLAP应用来说，分区能够显著提升查询性能，因为分析类查询往往需要处理大量数据。按时间进行分区，例如按月划分用户行为数据，可以使得查询只需扫描相关分区，从而提高效率。

然而，在OLTP应用中，使用分区则需更为谨慎。这类应用通常不会查询大表中超过10%的数据，而是通过索引快速检索少量记录。例如，对于包含1000万条记录的表，如果查询使用了辅助索引但未涉及分区键，可能导致性能下降。原本在单个B+树中3次逻辑IO就能完成的操作，在10个分区的情况下可能需要(3+3)*10次逻辑IO（分别访问聚集索引和辅助索引）。

因此，在OLTP应用中采用分区表时，务必进行充分的性能测试和优化。

为了便于开发者观察SQL查询对分区的利用情况，可以使用EXPLAIN PARTITIONS语句与SELECT查询结合，从而清晰地看到哪些分区被查询涉及。

三、分区表应用场景

1、日志表改造

https://blog.csdn.net/qg_42768234/article/details/141028690